Lab: Conflict Resolution During Merge

Objective

- Understand how to resolve conflicts that occur when multiple users modify the same file

Commands

1. Verify whether master is in a clean state.

```
$ git status
```

2. Create a branch GitWork and add a file hello.xml.

```
$ git branch GitWork
$ git checkout GitWork
$ echo "<message>Hello from GitWork</message>" > hello.xml
```

3. Update the content of hello.xml and observe the status.

```
$ echo "<message>Updated in GitWork</message>" > hello.xml
$ git status
```

4. Commit the changes to the branch.

```
$ git add hello.xml
$ git commit -m "Added hello.xml in GitWork"
```

5. Switch to master.

```
$ git checkout master
```

6. Add a file hello.xml to master with different content.

```
$ echo "<message>Hello from Master</message>" > hello.xml
```

7. Commit the changes to master.

```
$ git add hello.xml
$ git commit -m "Added hello.xml in master"
```

8. Observe the log.

```
$ git log --oneline --graph --decorate --all
```

9. Check differences using Git diff.

```
$ git diff master GitWork
```

10. View visual differences using P4Merge.

```
$ git difftool master GitWork
```

11. Merge the branch into master.

```
$ git merge GitWork
```

12. Observe the Git markup.

```
$ cat hello.xml
```

13. Resolve the conflict using a 3-way merge tool.

```
$ git mergetool
```

14. Commit the resolved changes.

```
$ git add hello.xml
$ git commit -m "Resolved merge conflict"
```

15. Check the status and add backup files to .gitignore.

```
$ git status
$ echo "*.bak" >> .gitignore
```

16. Commit the changes to .gitignore.

```
$ git add .gitignore
$ git commit -m "Added backup files to .gitignore"
```

17. List all available branches.

```
$ git branch -a
```

18. Delete the merged branch.

```
$ git branch -d GitWork
```

19. Observe the log.

```
$ git log –one --line --graph --decorate
```